VASTRA

PHASE 1 — SYSTEM DESIGN DOCUMENT

Design Review Submission | 19 October 2026

Amay Sharma UID: 25LBCS3018	**Kumar Shailendra** UID: 25LBCS3293

Prepared for the Integrated Full-Stack Project Design Review

Document Contents

1. Project Overview
2. Scope and Features
3. Functional Requirements
4. Non-Functional Requirements
5. Proposed Technology Stack
6. High-Level Architecture and Data Flow
7. API Design (Proposed)
8. Relational Database Design (3NF Plan)
9. Data Classification and DP-900 Plan
10. Cloud Security and Threat Model
11. Design Trade-Off
12. Assumptions, Risks, and Open Decisions
13. Phase 1 Deliverables and Next Steps

Status: Proposed design. Technology choices and integrations must be confirmed before implementation.

1. Project Overview

Project summary: Vastra is a clothing discovery and comparison web application intended to help shoppers search for apparel across supported online retailers, compare available product information, and make more informed buying decisions without visiting each retailer separately.

Problem statement: Online clothing shoppers often need to search several retailer websites separately to compare styles, prices, sizes, and product details. Product information is presented differently across sites, which makes comparison time-consuming. Customer reviews can also be difficult to access consistently, and a review should not be shown when the system cannot retrieve it from a source that can be identified. Vastra aims to bring product results into one interface, make comparisons easier, provide useful filters and saved items, and display customer reviews only when review data is actually available from a supported source. It will not fabricate reviews or present unavailable reviews as real.

Target users: People shopping online for clothing who want to discover products across supported retailers, compare options, and review available product information before opening the retailer's product page.

2. Scope and Features

Core features: 1) Search clothing products across supported retailers. 2) Show product cards with available title, image, price, retailer, and product link. 3) Filter and sort search results. 4) Compare product details across results. 5) Display real customer reviews only when a supported source returns them; otherwise, show no review section or a clear “reviews unavailable” message. 6) Save products to a wishlist for signed-in users.

Supporting features: User registration and login; role-based access for user and administrator; product detail view; retailer source links; search history where enabled; basic usage analytics; and an admin view for monitoring supported sources and failed fetches.

Out of scope for Phase 1: Guaranteed coverage of every retailer; guaranteed review availability for every product; automatic purchasing; payment processing; claiming that scraped or aggregated data is complete when the source does not provide it. Retailer coverage will depend on permitted, technically reliable sources.

3. Functional Requirements

FR-01 Search: Users can enter a product name or clothing keyword and receive matching products from the configured sources.

FR-02 Product listing: The system displays available product attributes and a link to the retailer page. Missing values are omitted or labelled as unavailable, not invented.

FR-03 Filter and sort: Users can filter or sort by available fields such as retailer, price, category, or rating when the source provides that field.

FR-04 Compare: Users can select products and compare their available attributes in a consistent layout.

FR-05 Reviews: The system shows review text, rating, and reviewer/date fields only when returned by a supported source. The source and retrieval time should be stored where practical. If none are returned, no fake reviews are shown.

FR-06 Wishlist: Authenticated users can save and remove products from their personal wishlist.

FR-07 Authentication and roles: Users can register, log in, and access only the functions permitted for their role. Administrators can manage source configuration and view operational logs.

FR-08 Activity logging: Security-relevant events and source-fetch failures are logged without storing passwords or unnecessary personal data.

4. Non-Functional Requirements

Security: Use HTTPS in deployment, securely hash passwords, store secrets outside source control, validate inputs, use parameterized SQL, and enforce server-side authorization.

Performance: Return useful search feedback promptly; use bounded timeouts and caching for repeat searches where appropriate. External source failures must not hang the entire request.

Reliability: If one retailer source fails, results from other available sources should still be returned where possible. Errors should be logged and communicated clearly.

Usability: Use consistent product cards, clear comparison fields, responsive layouts, and explicit handling of unavailable prices or reviews.

Maintainability: Keep retailer/source connectors behind a common service interface so a source can be changed without rewriting the whole application.

Privacy: Use dummy or public data for academic demonstration. Collect only necessary account information and do not store secrets or sensitive data in logs.

5. Proposed Technology Stack

Frontend: React with a responsive CSS framework (proposed).

Backend: Node.js with Express REST API (proposed).

Relational database: Azure SQL Database for users, product metadata, wishlist, searches, and review-source metadata.

Non-relational/object storage: Azure Blob Storage for product images or raw source response snapshots only when storage is permitted and needed; avoid copying unnecessary retailer content.

Analytics: SQL views and a lightweight Power BI report or equivalent report based on aggregated usage and source-health data.

Deployment: Azure-hosted web/API application using free-tier or university-provided resources where available.

Note: This stack is a proposal for the design review, not a claim that the application is already deployed. Confirm actual framework and hosting choices before implementation.

6. High-Level Architecture and Data Flow

Architecture: Browser / React UI → HTTPS REST API → application services (authentication, search orchestration, comparison, wishlist, review handling) → source connectors for supported retailers/APIs → normalisation layer → Azure SQL Database. Optional object storage holds permitted files or semi-structured source data. Aggregated operational data feeds the analytics report.

Request flow: 1) A user submits a search. 2) The API validates the request and applies rate limits. 3) The search service queries configured retailer connectors with timeouts. 4) Connector results are normalised into a common product shape. 5) The service removes duplicates where identifiers permit and stores only the fields needed by the app. 6) The API returns products and source links. 7) Review data is attached only if the connector returns identifiable review records; otherwise reviews are omitted. 8) The UI displays the results and allows comparison or wishlist actions.

Architecture diagram

```
USER BROWSER (React UI)\n        | HTTPS\n        v\nREST API (Node.js / Express)\n        |-- Authentication + RBAC\n
```

7. API Design (Proposed)

POST /api/auth/register: Create a user account after validation.

POST /api/auth/login: Authenticate a user and issue a secure session/token.

GET /api/products/search?q={query}&retailer={name}&minPrice={n}&maxPrice={n}&sort={field}: Search configured sources and return normalised product results.

GET /api/products/{productId}: Return stored product details and retailer source link.

GET /api/products/compare?ids={id1,id2,...}: Return comparable attributes for selected products.

GET /api/products/{productId}/reviews: Return reviews only if available from a supported source; otherwise return an empty list and availability status.

GET /api/wishlist: Return the signed-in user's saved products.

POST /api/wishlist: Save a product to the signed-in user's wishlist.

DELETE /api/wishlist/{wishlistItemId}: Remove a saved product.

GET /api/admin/source-health: Admin-only summary of connector status and failures.

Security note: All routes require input validation. Wishlist and admin endpoints require server-side authorization. Exact request/response schemas will be documented during implementation.

8. Relational Database Design (3NF Plan)

Table plan: The proposed schema has more than five related tables and separates user, retailer, product, search, wishlist, and review records to reduce duplication.

Users: user_id PK; name; email UNIQUE; password_hash; role; created_at.

Retailers: retailer_id PK; retailer_name UNIQUE; base_url; is_active.

Products: product_id PK; retailer_id FK → Retailers; external_product_id; title; category; price; currency; product_url; image_url; last_seen_at. Add a uniqueness constraint on (retailer_id, external_product_id) where supported.

Searches: search_id PK; user_id FK → Users nullable; query_text; created_at; result_count.

WishlistItems: wishlist_item_id PK; user_id FK → Users; product_id FK → Products; saved_at; UNIQUE(user_id, product_id).

Reviews: review_id PK; product_id FK → Products; source_review_id nullable; rating nullable; review_text; reviewer_label nullable; review_date nullable; retrieved_at; source_url. Store only when returned by an identifiable supported source.

ActivityLogs: activity_id PK; user_id FK → Users nullable; event_type; resource_type; resource_id nullable; created_at; outcome. Do not store passwords, tokens, or unnecessary personal data.

Relationships: Retailers 1—many Products; Users 1—many Searches; Users 1—many WishlistItems; Products 1—many WishlistItems; Products 1—many Reviews; Users 1—many ActivityLogs.

Constraints and indexes: Primary keys on all tables; foreign keys for relationships; UNIQUE email; UNIQUE retailer name; unique retailer/external product ID; unique user/product wishlist pair; CHECK constraints for non-negative prices and valid ratings where ratings are numeric. Index product title/category/retailer/price columns used in search and foreign-key columns used in joins.

Required SQL demonstrations: Implementation should include joins, at least one view (for example, product search/retailer summary), one stored procedure or trigger, and one transaction (for example, saving a wishlist item with related audit logging).

9. Data Classification and DP-900 Plan

Structured data: Users, retailers, products, searches, wishlist items, reviews, and activity logs stored in Azure SQL Database.

Semi-structured data: Optional JSON connector responses or source-health payloads stored in Blob Storage only if needed and permitted.

Unstructured data: Product image files or other permitted files stored in Azure Blob Storage where necessary; otherwise keep source image URLs rather than copying images.

Analytics: A database view and a small Power BI report can show search volume, products returned by retailer, source-fetch success/failure, and review availability. Use aggregated metrics and dummy/public data.

Evidence for later review: Capture Azure resource configuration screenshots and include the data-classification note and analytics report at the security-check/final milestones.

10. Cloud Security and Threat Model

Unauthorized access: Require authentication; enforce role-based authorization on the server; test user and admin permissions.

Password exposure: Store password hashes using a modern password-hashing algorithm and never log credentials.

SQL injection: Use parameterized queries or ORM-safe query methods; validate types, lengths, and allowed sort fields.

Cross-site scripting: Escape/encode untrusted text at output and avoid injecting raw HTML from retailer descriptions or reviews.

Secret leakage: Use environment variables or a managed secret store; commit only example configuration with placeholder values.

Database exposure: Restrict database network access using firewall/network rules; use least-privilege database credentials.

Transport interception: Use HTTPS for the deployed app and secure cookie/token settings.

Malicious or broken source data: Treat retailer results and reviews as untrusted input; normalise and validate fields before storing or rendering.

Abuse and availability: Set request timeouts, bounded concurrency, rate limits, and safe error handling for source connectors.

Activity logging: Record security-relevant events and connector failures; restrict log access and avoid sensitive values.

11. Design Trade-Off

Chosen approach: Use a modular monolith for the initial application rather than microservices.

Reasoning: A small student team can build, test, and deploy one application more quickly. Search, comparison, authentication, and wishlist logic can still be separated into modules. This reduces deployment and operational complexity while keeping a future path open to split out source connectors if traffic or reliability needs justify it.

Cost: External retailer APIs or scraping providers may have usage limits and can change their responses. The system should begin with a small number of permitted, reliable sources and gracefully handle missing data instead of promising universal coverage.

Caching trade-off: Caching can reduce repeated source calls and improve response time, but prices and availability can become stale. If implemented, cached results should include a retrieval timestamp and a short, documented expiry.

12. Assumptions, Risks, and Open Decisions

Assumptions: The application is for product discovery and comparison, not checkout. Product and review availability depends on each source. Reviews are never generated or invented.

Risks: Retailer page changes, source access restrictions, rate limits, inconsistent product identifiers, incomplete prices, and unavailable reviews.

Mitigations: Use a common connector interface, source links and timestamps, timeouts, validation, clear unavailable states, and a small initial retailer set.

To confirm before implementation: Final framework and hosting choice; permitted retailer data sources; exact fields each source returns; whether user accounts are needed in the first demo; and which Azure non-relational store will be used.

13. Phase 1 Deliverables and Next Steps

Design review package: This design document; an ER diagram based on the proposed tables; and this threat model section, which can also be extracted into a one-page threat model.

After design approval: Implement the schema and API; build one core feature end to end; add a view, stored procedure or trigger, transaction, joins, and indexes; deploy to Azure; add the non-relational store and analytics report; complete and test the security checklist.

Academic integrity: The document describes a proposed design. The team should understand and be able to explain each component, and should update the document if the implemented design differs.